

MLM CREATOR
A PROJECT REPORT
Submitted by
EESAM THANU BISHWANTH BAJI
(Reg. No. 23352020)
in partial fulfillment for the award of the degree
of
MASTER OF COMPUTER APPLICATIONS



DEPARTMENT OF COMPUTER SCIENCE
SCHOOL OF ENGINEERING AND TECHNOLOGY
PONDICHERY UNIVERSITY
PUDUCHERRY -605014
MAY 2025

BONAFIDE CERTIFICATE

This is to certify that this project work entitled “**MLM CREATOR**” is a Bonafide record of work done by **EESAM THANU BISHWANTH BAJI, Reg. No. 23352020**, in partial fulfillment of the requirements for the degree of **Master of Computer Applications** in the Department of Computer Science, School of Engineering and Technology of Pondicherry University.

This work has not been submitted elsewhere for the award of any other degree to the best of our knowledge.

PROJECT GUIDE

Dr. S.L JAYALAKSHMI

Assistant Professor

Department of Computer Science

School of Engineering & Technology

Pondicherry University

HEAD OF THE DEPARTMENT

Dr. S.K. V. JAYAKUMAR

Professor & Head

Department of Computer Science

School of Engineering & Technology

Pondicherry University

Submitted for the Viva- Voce Examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

I would like to take this opportunity to express my heartfelt gratitude to all those who have supported and guided me throughout the development of the MLMCreator web application project. This project has been an insightful journey, and its successful completion would not have been possible without the encouragement, cooperation, and technical input from many individuals.

First and foremost, I extend my sincere thanks to Pondicherry University and the Department of Computer Applications for providing a solid academic foundation and the environment to explore and innovate.

I am especially grateful to my project supervisor, **Dr. S.L. Jayalakshmi**, whose expert guidance, timely suggestions, and constant encouragement were instrumental throughout the design and implementation of this project. Their experience and attention to detail significantly helped improve the quality and clarity of my work.

I wish to express my sincere thanks to **Dr. S.K.V. JAYAKUMAR**, Professor and Head, Department of Computer Science, Pondicherry University, for his kind advice and constant encouragement during this project work.

My gratitude also extends to all the faculty members and non-teaching staff of the Department of Computer Science, for all the assistance they rendered so willingly, to help me in completing my project report.

EESAM THANU BISHWANTH BAJI

TABLE OF CONTENTS

TITLE	PAGE NUMBER
ABSTRACT	2
INTRODUCTION	3
1.1. About the Project	4
SYSTEM ANALYSIS	5
2.1. Existing System	6
2.2. Proposed System.....	7
2.3. Requirement Analysis.....	8
2.4. Hardware Requirements	9
2.5. Software Requirements.....	10
2.6. Use Case Analysis	11
SYSTEM DESIGN	12
3.1. UI Design.....	13
3.2. Architectural Design	13
3.3. Database Design.....	14
3.4. UML Diagrams.....	15
Use Case Diagram	15
Class Diagram.....	16
Sequence Diagram	17
Activity Diagram	18
Data Flow Diagram (DFD)	19
IMPLEMENTATION	20
4.1. Frontend Implementation	20
4.2. Backend Implementation	22

TESTING	24
5.1. Unit Testing.....	24
5.2. Functional Testing	25
5.3. Integration Testing	26
5.4. System Testing	27
CONCLUSION	28
6.1. Foreseeable Enhancements	29
APPENDIX A - CODES.....	30
APPENDIX B - SCREEN SHOTS	33
BIBLIOGRAPHY.....	36

ABSTRACT

MLMCreator is an end-to-end, interactive, and user-friendly web application designed to simplify the process of building, training, evaluating, and deploying machine learning (ML) models without requiring users to write a single line of code. Built using Streamlit, MLMCreator democratizes machine learning by offering a guided and modular interface that caters to both novice users and professionals who seek rapid prototyping and deployment of ML models from structured datasets.

The application features a secure login system and supports seamless dataset uploads in various formats including CSV, Excel, JSON, ARFF, and more. Uploaded datasets are stored in a PostgreSQL database and made available to the authenticated user throughout the modelling pipeline. The preprocessing stage is designed to emulate a Jupyter Notebook experience, allowing users to apply transformations through interactive, code-free steps such as handling missing values, encoding categorical variables, feature scaling, and feature selection.

MLMCreator intelligently recommends a suitable task type, classification or regression based on the nature of the target column. It also suggests optimal train-test split ratios depending on dataset size. A wide range of traditional machine learning algorithms is available for training, including Random Forest, Logistic Regression, SVM, KNN, and Gradient Boosting. Additionally, the application supports image-based classification tasks using Convolutional Neural Networks (CNNs), trained directly from user-supplied image datasets containing file paths.

The training module evaluates models using appropriate metrics such as accuracy, F1-score, precision, recall for classification, and MAE, RMSE, R^2 for regression. Visualizations like confusion matrices and scatter plots help interpret model performance. All trained models, along with their evaluation metrics, are saved in the backend, allowing users to revisit, download, or delete them.

The Models page summarizes all trained models by dataset and task type, highlighting the best-performing models and enabling selection for downstream prediction. The Predict page allows users to interactively provide new inputs, map label outputs to user-friendly classes, and receive model predictions in real time.

Overall, MLMCreator bridges the gap between data exploration and machine learning deployment by integrating data management, preprocessing, model training, evaluation, and inference in a unified, web-based interface. It is a valuable tool for data scientists, students, and business analysts looking to accelerate ML workflows and make data-driven decisions effortlessly.

CHAPTER 1

INTRODUCTION

The growing accessibility of data has propelled machine learning (ML) into mainstream applications across industries such as healthcare, finance, marketing, and education. However, while ML has the potential to unlock powerful insights and predictive capabilities, building and deploying ML models remains a complex task that often requires programming expertise, mathematical understanding, and domain-specific knowledge. This technical barrier prevents many individuals especially non-programmers, analysts, and domain experts from leveraging ML to its full potential.

MLMCreator is designed to bridge this gap by offering a no-code, end-to-end platform that simplifies the machine learning lifecycle from data ingestion to model deployment. Built using Streamlit, a Python-based web framework for data applications, MLMCreator provides an intuitive and interactive interface where users can upload datasets, preprocess data, train models, visualize results, and perform predictions without writing any code. This enables users with little or no technical background to build robust machine learning models using a guided and visual workflow.

The platform supports various data formats including CSV, Excel, JSON, ARFF, and more, ensuring wide compatibility with real-world datasets. Once uploaded, datasets are stored securely in a PostgreSQL database, allowing seamless access across sessions. The preprocessing module is structured like a step-by-step notebook, offering transformations such as missing value imputation, encoding of categorical variables, feature scaling, and selection. Users make choices through interactive widgets, and all changes are tracked to maintain transparency.

MLMCreator includes a model training engine that intelligently recommends the type of ML task classification or regression based on the characteristics of the target variable. Users can choose from a set of powerful models including Random Forest, Logistic Regression, Support Vector Machines (SVM), K-Nearest Neighbours (KNN), and Gradient Boosting.

Model evaluation is comprehensive, offering metrics like accuracy, precision, recall, F1-score, MAE, RMSE, and R^2 , along with visualizations such as confusion matrices, ROC curves, scatter plots, and residual graphs. Trained models are saved with their metrics in a structured database and can be downloaded or selected for prediction through a dedicated interface.

1.1. About the Project

The MLMCreator web application is a comprehensive, modular platform designed to simplify and automate the machine learning workflow for tabular datasets. Targeted at students, educators, beginners in machine learning, and analysts, the project focuses on providing an end-to-end solution from data upload to model training and prediction in a seamless, intuitive, and code-free environment.

The core idea behind MLMCreator is to enable users to work through the machine learning lifecycle without requiring deep technical expertise. After registering and logging in, users can upload their datasets in CSV format. The application then allows users to explore and analyse their data through a dedicated "Analyse" page, offering insights like null value counts, basic statistics, and graphical visualizations.

Following analysis, users can preprocess the dataset using simple, guided options such as null value handling, encoding categorical data, scaling, and feature selection. Only after a successful dataset upload does the system allow access to the Preprocessing and Training modules, ensuring a smooth and logical flow of operations.

On the Train page, users can select various classification or regression models, set training parameters, and evaluate model performance with metrics and visualizations. Trained models are saved in the PostgreSQL database along with metadata like model type, metrics, timestamps, user ID, and dataset ID, providing an organized way to track model history.

In the Models page, users can view a list of their trained models, along with performance metrics. They can then select a model to proceed to the Predict page, where new predictions can be made on fresh data inputs. Importantly, users cannot access the Predict page without selecting a trained model first, enforcing a logical sequence and avoiding errors.

The backend of the system is built using Flask for handling APIs, user authentication, dataset storage, model training, and prediction logic. The frontend is developed using React.js, with a focus on creating an interactive, notebook-like user interface that is clean, responsive, and easy to navigate. Styling is done without heavy frameworks like Tailwind to keep the UI lightweight and customizable.

Overall, MLMCreator aims to bridge the gap between raw data and machine learning deployment by providing an easy-to-use, secure, and efficient platform. It offers persistent storage for datasets and models, session management, stepwise interaction, and visual feedback, making machine learning accessible to a broader range of users without requiring extensive coding knowledge.

CHAPTER 2

SYSTEM ANALYSIS

Machine learning (ML) is a powerful tool that enables systems to learn from data and make informed decisions or predictions without being explicitly programmed. Over the past decade, it has become an essential component in domains such as healthcare diagnostics, financial forecasting, customer segmentation, fraud detection, and more. Despite its vast potential, the application of machine learning remains largely inaccessible to many individuals and small organizations due to a significant learning curve and reliance on technical expertise.

The current landscape of ML development typically involves several complex steps: data collection, preprocessing, model selection, training, evaluation, and deployment, each of which requires proficiency in programming languages (like Python or R), familiarity with machine learning libraries (such as scikit-learn, TensorFlow, or PyTorch), and an understanding of statistical and mathematical concepts. This technical barrier excludes non-programmers, subject matter experts, educators, and analysts who possess domain knowledge but lack the skills to implement ML solutions on their own.

Moreover, existing ML platforms and frameworks often either cater to experienced data scientists or provide limited, black-box solutions with minimal customization and interpretability. In many cases, users are either overwhelmed by complexity or unable to control or understand how models make decisions. This gap between usability and control has created a growing demand for tools that make ML more approachable, interpretable, and manageable.

In addition, the lack of integrated systems that streamline the end-to-end ML workflow into a single, cohesive experience further hinders adoption. Users often must switch between multiple tools for data cleaning, model training, evaluation, and deployment, which adds inefficiencies and increases the risk of errors and data loss.

Therefore, there is a critical need for a user-friendly, no-code platform that allows users to perform all essential ML tasks within a single interface. The solution should guide users step-by-step, ensure data transparency, support multiple model types, provide visual feedback, and enable real-time predictions all without requiring programming knowledge.

MLMCreator addresses this gap by offering a full-featured, intuitive web application for automated machine learning. It empowers users to transform raw data into trained models and actionable insights through an interactive and visual interface. By simplifying the entire ML pipeline, MLMCreator aims to democratize machine learning and make it accessible to a broader audience.

2.1. Existing System

In the current technological landscape, there are several tools and platforms available for developing and deploying machine learning models. These systems range from powerful libraries and frameworks for developers to simplified, GUI-based tools designed for less technical users. However, each of these systems has its own set of limitations that hinder accessibility, usability, or flexibility, especially for beginners or domain experts without programming knowledge.

Popular open-source libraries such as Scikit-learn, TensorFlow, Keras, and PyTorch are widely used in professional ML pipelines. These tools offer robust functionalities for data processing, model training, and evaluation. However, they require a high level of proficiency in programming (mostly Python), knowledge of machine learning concepts, and familiarity with command-line interfaces or Jupyter Notebooks. This makes them largely unsuitable for non-programmers or users from non-technical backgrounds.

Weka supports data files in the Attribute-Relation File Format (.ARFF) and allows importing CSV files. Users can apply a variety of filters for data transformation and cleaning, making it a flexible platform for basic data preparation. It includes numerous algorithms such as decision trees, support vector machines, Naïve Bayes, k-nearest neighbours, and neural networks, which can be applied with minimal coding knowledge.

Despite its strengths, Weka has some limitations. It lacks support for modern web-based interfaces, and all operations are performed on the local machine. This restricts its scalability and real-time collaboration features. Weka is best suited for small to medium-sized datasets, as large datasets can cause performance issues. It also lacks direct integration with contemporary data science tools and frameworks such as Python's scikit-learn, pandas, and deep learning libraries.

Weka does not support user authentication or multi-user environments, which limits its use in production-grade applications. The models built in Weka are not readily deployable for inference in web or cloud applications without significant manual intervention. It also does not provide seamless support for versioning, persistence, or visualization enhancements common in modern web-based tools.

There are also code-free ML tools like Teachable Machine and Lobe AI, which are extremely beginner-friendly but focus primarily on image classification tasks and lack support for broader use cases such as regression, tabular classification, or CSV-based predictions. They also provide limited data preprocessing capabilities, feature engineering, and metric visualization features that are critical for a comprehensive ML workflow.

In summary, existing systems either cater to expert users with technical backgrounds or offer simplified solutions that sacrifice control, flexibility, or support for real-world use cases. There is a noticeable lack of a unified, no-code, customizable, and intuitive machine learning platform tailored for non-programmers who want full control and visibility over their end-to-end ML journey.

2.2. Proposed System

MLMCreator is a modern, web-based machine learning workflow automation tool designed to streamline the end-to-end data science process for tabular datasets. Unlike traditional tools like Weka, MLMCreator brings the entire ML pipeline to a user-friendly browser interface, providing a collaborative and scalable solution tailored to students, educators, analysts, and beginner ML practitioners.

The application enables users to securely register and log in, ensuring user-specific dataset and model management. Upon logging in, users can upload tabular datasets (in CSV format), view them in an interactive table, and perform automated exploratory data analysis (EDA), including summary statistics and data visualizations. The interface facilitates preprocessing steps such as handling null values, encoding categorical variables, feature selection, and normalization through an intuitive and code-free interface.

A key feature of MLMCreator is its modular workflow, which guides users step-by-step through the machine learning process. After preprocessing, users can train a variety of machine learning models including classification and regression algorithms using popular libraries like scikit-learn. During training, users can compare models based on metrics such as accuracy, precision, recall, F1 score, and RMSE, with results presented through clean plots and tables.

All trained models are saved in a PostgreSQL database along with associated metadata, including dataset ID, user ID, model type, performance metrics, and timestamps. The Models page allows users to review their saved models, while the Predict page enables them to load a chosen model and make predictions on new data directly through the web interface.

Access to the Preprocessing and Train pages is restricted until a dataset is uploaded, ensuring a logical and error-free workflow. Similarly, the Predict page is disabled until a model is selected, enforcing proper sequence adherence.

The backend is developed using Python's Flask framework, offering secure API endpoints for data manipulation, model training, prediction, and storage. The frontend is built with React and styled for responsiveness and clarity, offering a notebook-like experience without the need for writing code. The app runs within an Anaconda environment, supporting streamlined Python package management and compatibility.

2.3. Requirement Analysis

MLMCreator is a web-based application designed to provide an end-to-end machine learning model development experience without requiring the user to write code. Its primary objective is to simplify the machine learning workflow for users such as data analysts, students, and business professionals who may not have programming skills but still wish to explore data-driven insights. The application integrates dataset handling, preprocessing, model training, evaluation, and prediction into a unified, interactive interface built with Streamlit.

A core functional requirement is secure user authentication. The system must support user registration, login, and logout, and ensure that each user has access only to their own datasets and models. Once logged in, users can upload datasets in various formats, including CSV, Excel, JSON, XML, ARFF, and others. These datasets are validated, converted to a standardized format (CSV), previewed, and stored in a PostgreSQL database.

Another major requirement is an interactive preprocessing interface that mimics the stepwise execution of a Jupyter Notebook. Users should be able to perform operations like dropping columns, handling missing values, encoding categorical features, and scaling numerical data through a visual interface. Each transformation should show real-time feedback, and users should be able to apply multiple transformations sequentially.

For model training, the application should intelligently suggest whether the task is classification or regression based on the selected target column. It must also recommend an appropriate test/train split ratio depending on the dataset size. Users can then select from traditional models such as Logistic Regression, Random Forest, SVM, KNN, and Convolutional Neural Networks (CNN). The training process must display metrics for both training and test datasets, but only the test metrics are saved to the database.

The system should also visualize evaluation results through confusion matrices, ROC curves, feature importance plots, and error visualizations like residuals or predicted vs. actual plots. All trained models are saved in a dedicated models/ directory, and their metadata is logged in the database.

Users must be able to view, download, delete, and select models for prediction from the Models page. For prediction, the app should allow users to input values for features and display results in a clear and interpretable format, including label mappings for classification.

Overall, MLMCreator must combine intuitive design, smart automation, and robust backend logic to deliver a fully interactive, user-friendly machine learning platform.

2.4. Hardware Requirements

Client Side (User System):

- Processor: Intel i3 or higher / AMD Ryzen 3 or higher
- RAM: Minimum 4 GB (8 GB recommended for smoother performance)
- Storage: At least 500 MB of free space for browser cache and downloads
- Display: 1280×720 resolution or higher
- Input: Keyboard, Mouse/Touchpad

Server Side (Deployment Machine / Development Environment):

- Processor: Intel i5/i7 or AMD Ryzen 5 or above
- RAM: Minimum 8 GB (16 GB recommended for large dataset/model handling)
- Storage: 50 GB SSD minimum (to store datasets, models, user data)

2.5. Software Requirements

Frontend:

- Browser: Google Chrome, Firefox, Microsoft Edge (latest versions)
- Framework: Streamlit 1.30.0 or above
- Language: HTML5, CSS3, JavaScript (used in Streamlit components)

Backend:

- Language: Python 3.9 or above
- Libraries:
 - Streamlit
 - Pandas, NumPy, Matplotlib, Seaborn
 - Scikit-learn, TensorFlow/PyTorch (for DL tasks)
 - SQLAlchemy, psycopg2 (PostgreSQL integration)
 - bcrypt (for authentication)
- Web Server: Gunicorn or Streamlit's internal server (for dev use)

Database:

- PostgreSQL 13 or above
- pgAdmin (optional, for DB GUI)

Environment & Tools:

- IDE: VS Code, PyCharm, or Jupyter Lab
- Package Manager: pip or conda
- Environment Manager: virtualenv or conda environments
- Git for version control

3. Network Requirements:

- Internet connection for model training on cloud resources, dataset download, and package installation

2.6. Use Case Analysis

The MLMCreator web application is designed to streamline the machine learning pipeline by offering an end-to-end, code-free experience for users to build and deploy models. The primary use case begins with user authentication. Users must log in or register before accessing the platform's functionalities. This ensures that all data uploaded, models trained, and results generated are securely linked to the specific user, maintaining privacy and organization across sessions. Session management is built in, and pages like Upload, Preprocess, Train, Models, and Predict are restricted if the user is not logged in.

The next major use case is dataset uploading. Users can upload a wide range of file types, including CSV, Excel (XLSX/XLS/ODS), ARFF, JSON, XML, TSV, TXT, binary formats like .dat, .db, and even spreadsheet-based formats. Upon upload, the application automatically converts non-CSV files to CSV format for standardization, previews the dataset, and saves it to a PostgreSQL database while linking it to the current user. This enables reusability and data integrity while simplifying the ingestion process for a diverse user base.

The preprocessing use case focuses on interactive data cleaning. Users can perform tasks like dropping columns, handling missing values, encoding categorical variables, and scaling features through a stepwise interface that mimics the structure of Jupyter Notebooks. Each step is visually presented as a cell, and users can apply transformations without writing code. These steps are saved in the session and applied sequentially, allowing for a smooth and repeatable preprocessing workflow.

Training is another core use case. The application intelligently suggests whether the problem is a classification or regression task based on the target column's properties. It also recommends an appropriate train-test split ratio depending on the dataset size. Users can select from machine learning models such as Random Forest, Logistic Regression, Support Vector Machines, KNN, SVR, and Gradient Boosting, as well as deep learning models. Once trained, the system evaluates the model using classification or regression metrics and presents visual plots for better interpretation. The model is saved in a dedicated models directory, and only the test set metrics are stored in the database.

Lastly, users can view all their trained models, grouped by dataset and task type. They can download models, delete them, or mark one for use in predictions.

In the Predict use case, users specify input data and receive model predictions with a user-friendly interpretation. This workflow completes a no-code, end-to-end ML journey.

CHAPTER 3

SYSTEM DESIGN

The system design of MLMCreator focuses on delivering a seamless, secure, and scalable web application that allows users to perform end-to-end machine learning tasks without writing code. The architecture follows a modular, layered design consisting of a user interface layer, an application logic layer, and a database layer. Each layer interacts with others through well-defined interfaces, ensuring maintainability, scalability, and ease of development.

At the front end, MLMCreator uses Streamlit, a powerful Python-based web framework, to provide a responsive and intuitive user interface. The application is structured into multiple pages using Streamlit's native multipage feature, including Home, Upload, Preprocess, Train, Models, and Predict. Navigation is managed through a sidebar, which dynamically adapts based on the user's authentication status. Session management is implemented to ensure that user context is preserved across page transitions and that only authorized users can access internal functionalities.

The application logic layer handles data processing, model training, and prediction workflows. Preprocessing operations are interactive and follow a Jupyter Notebook-like structure where each transformation (e.g., missing value handling, encoding, scaling) is presented as a step. These transformations are applied in order, stored in the session, and can be traced or modified by the user. The Train module recommends task type (classification or regression) and an optimal train-test split ratio based on the dataset's characteristics. Users select machine learning models, including Random Forest, SVM, Logistic Regression, KNN, Linear Regression, Gradient Boosting, and CNN. After training, performance metrics and plots (e.g., confusion matrix, ROC curve, residual plot) are displayed and interpreted to aid understanding.

Trained models are saved in a dedicated models directory on the server using Joblib and registered in a PostgreSQL database. Each model entry stores metadata, evaluation metrics, and file paths. The Models page allows users to view, download, delete, or mark models for future prediction. When a model is selected for prediction, its ID is stored in the session state and used in the Predict page. The prediction interface allows input of new feature values and provides output in a human-readable format, especially when target values are categorical.

At the backend, a PostgreSQL database stores users, datasets, models, and associated metrics. Tables are normalized and linked via foreign keys to maintain referential integrity. This structure supports robust querying and analytics across the platform.

Overall, MLMCreator's system design balances usability with technical depth, enabling a wide range of users from beginners to experts to build reliable machine learning models with ease and transparency.

3.1. UI Design

The user interface (UI) design of MLMCreator is centered around simplicity, clarity, and interactivity to provide an accessible experience for users with varying levels of expertise. Built using Streamlit, the UI features a clean, sidebar-based navigation that divides the application into logical sections: Home, Upload, Preprocess, Train, Models, and Predict. Each page guides the user through a step-by-step machine learning workflow, with forms, dropdowns, sliders, and visual feedback. The Preprocess page mimics a Jupyter Notebook interface, presenting preprocessing steps as interactive code-like cells that allow non-programmers to perform data cleaning intuitively. Visualizations such as charts, confusion matrices, and ROC curves are integrated using Plotly to make evaluation results easier to interpret. The interface is responsive, with real-time feedback, toast notifications, and validation prompts that ensure smooth user interaction. Overall, the UI promotes ease of use, while preserving the functionality and depth required for end-to-end machine learning model development.

3.2. Architectural Design

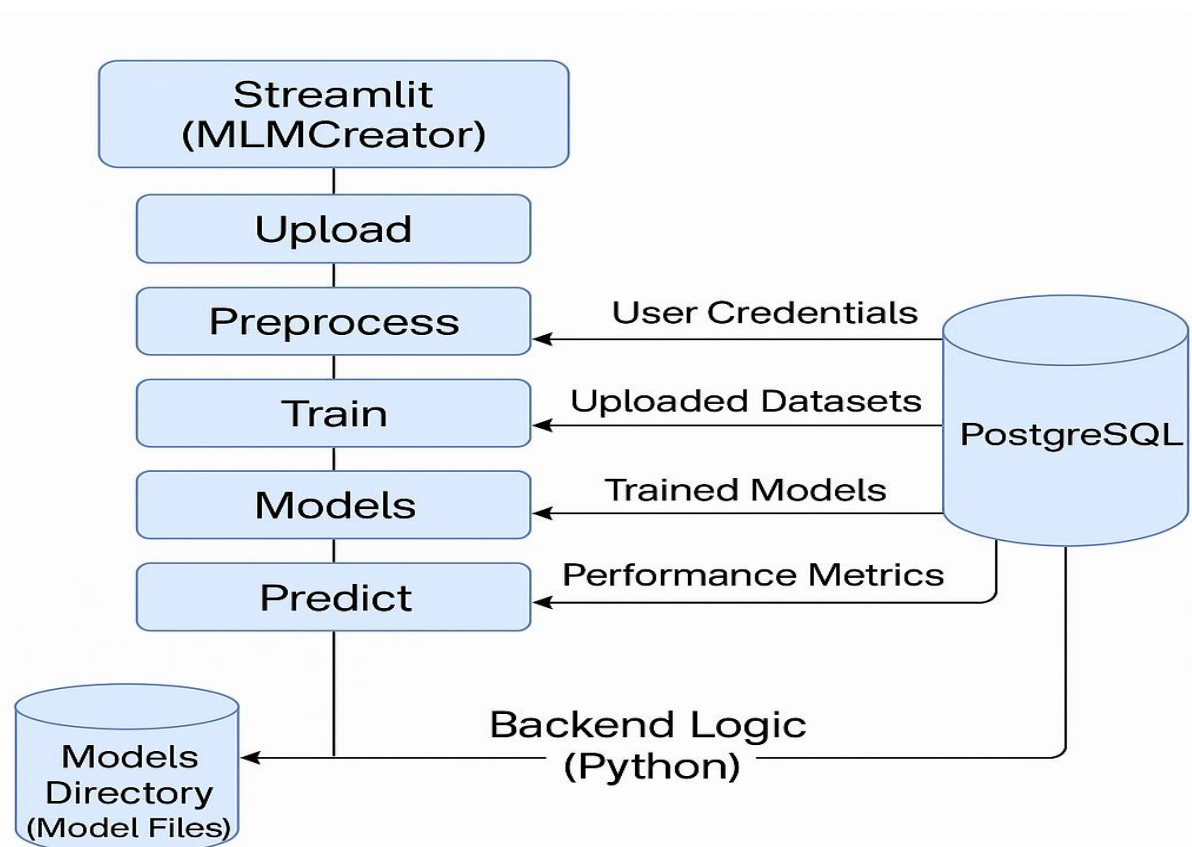


Figure 3.2.1 Architecture Design

3.3. Database Design

The database design for the MLMCreator web application is structured to ensure data integrity, scalability, and efficient access across multiple modules. At its core, the database is built using PostgreSQL and follows a relational schema that ties user activity to uploaded datasets, preprocessing steps, model training, and saved models. This modular design allows for stepwise interaction while ensuring that progress at each stage is stored persistently.

The users table stores credentials and profile information, with hashed passwords using bcrypt for security. The datasets table links each dataset to its user through a foreign key, storing metadata such as upload time, file name, and data structure. As users progress, the preprocessing table stores transformation steps applied to datasets, such as null handling or encoding.

The models table holds information about trained models, including algorithm used, hyperparameters, performance metrics (like accuracy or RMSE), training time, and the path to the saved model file. It also links back to both users and datasets for traceability. Additional tables like model_metrics and session_logs ensure detailed tracking and support auditability. This design facilitates a seamless experience across the ML pipeline while maintaining a clean, normalized, and query-efficient structure.

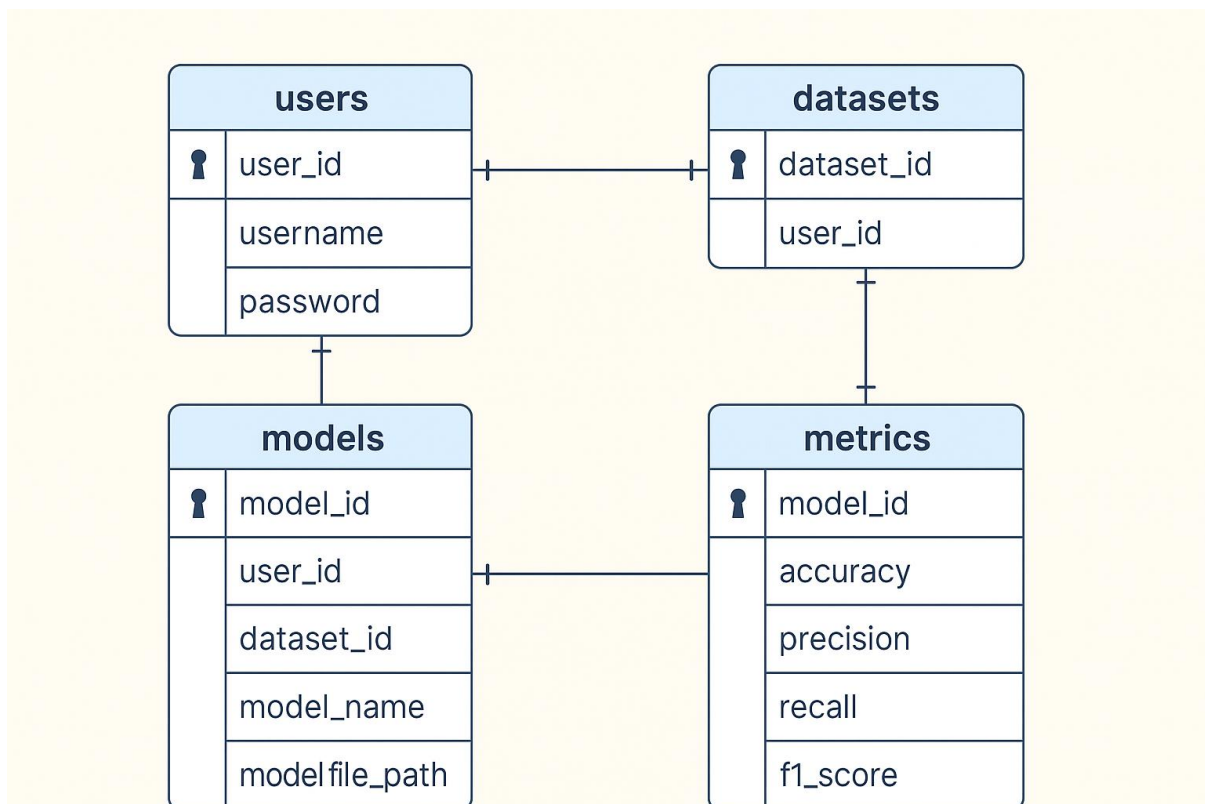


Figure 3.3.1 Database Design

3.4. UML Diagrams

Use Case Diagram

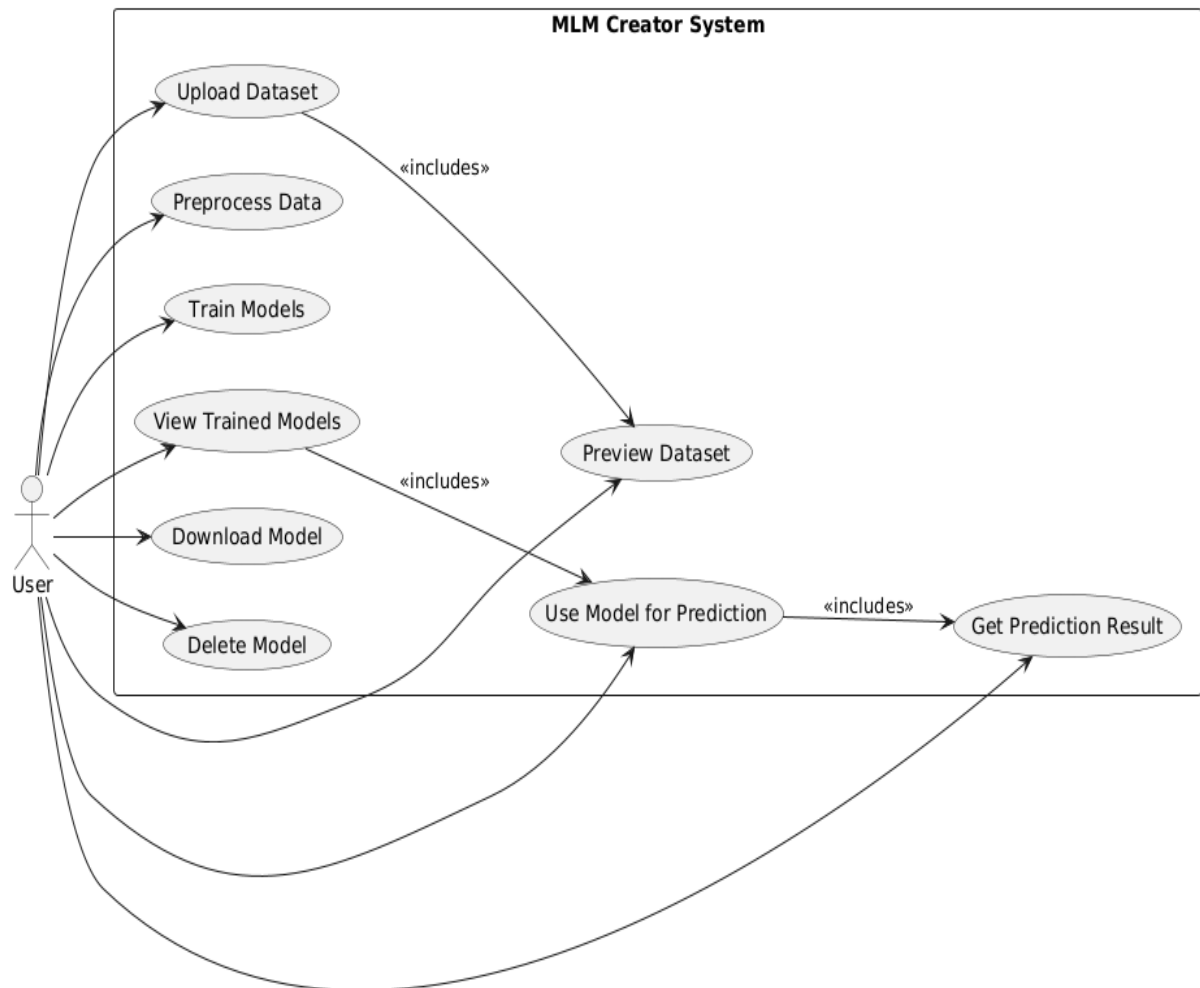


Figure 3.4.1 Use Case Diagram

Class Diagram

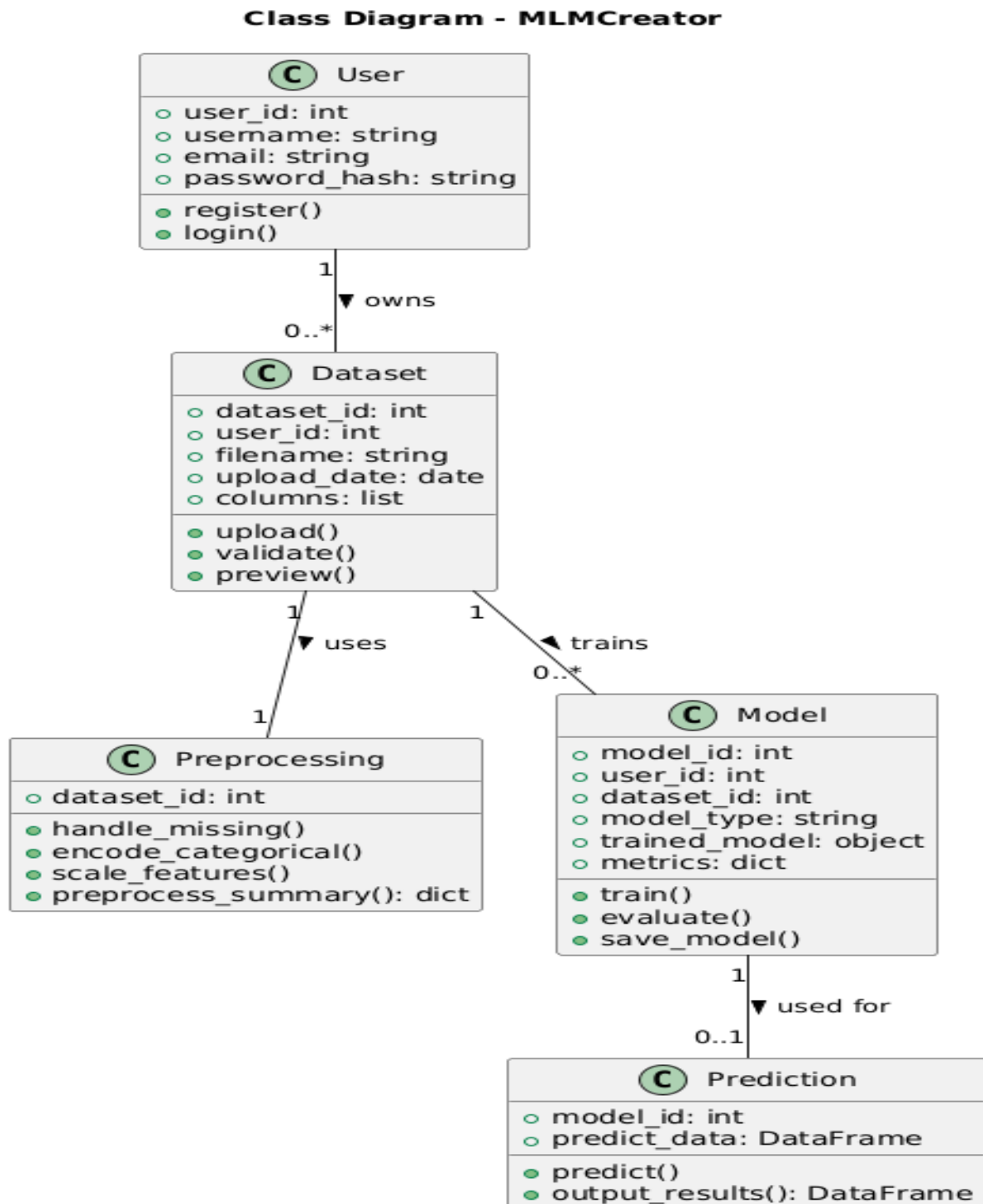


Figure 3.4.2 Class Diagram

Sequence Diagram

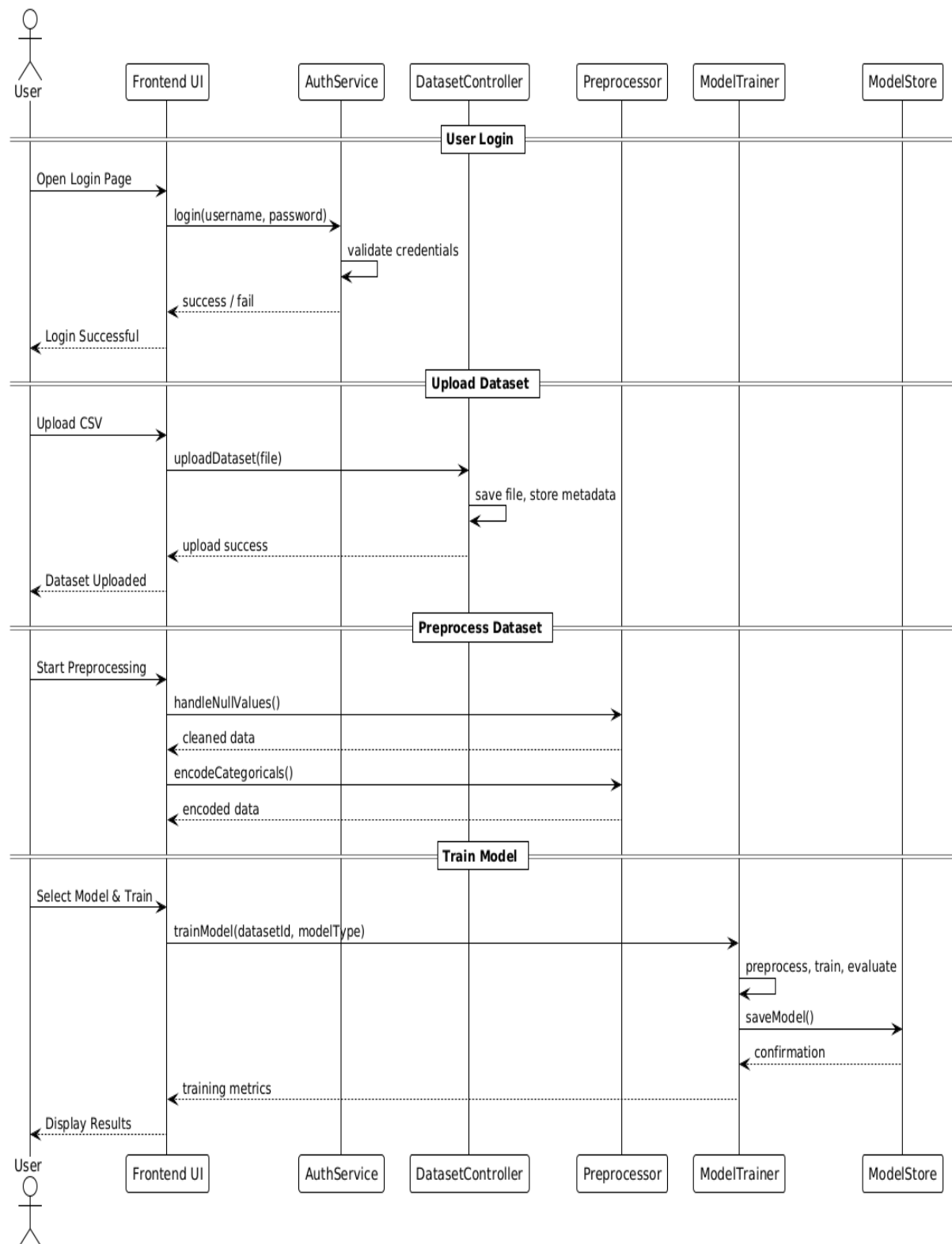


Figure 3.4.3 Sequence Diagram

Activity Diagram

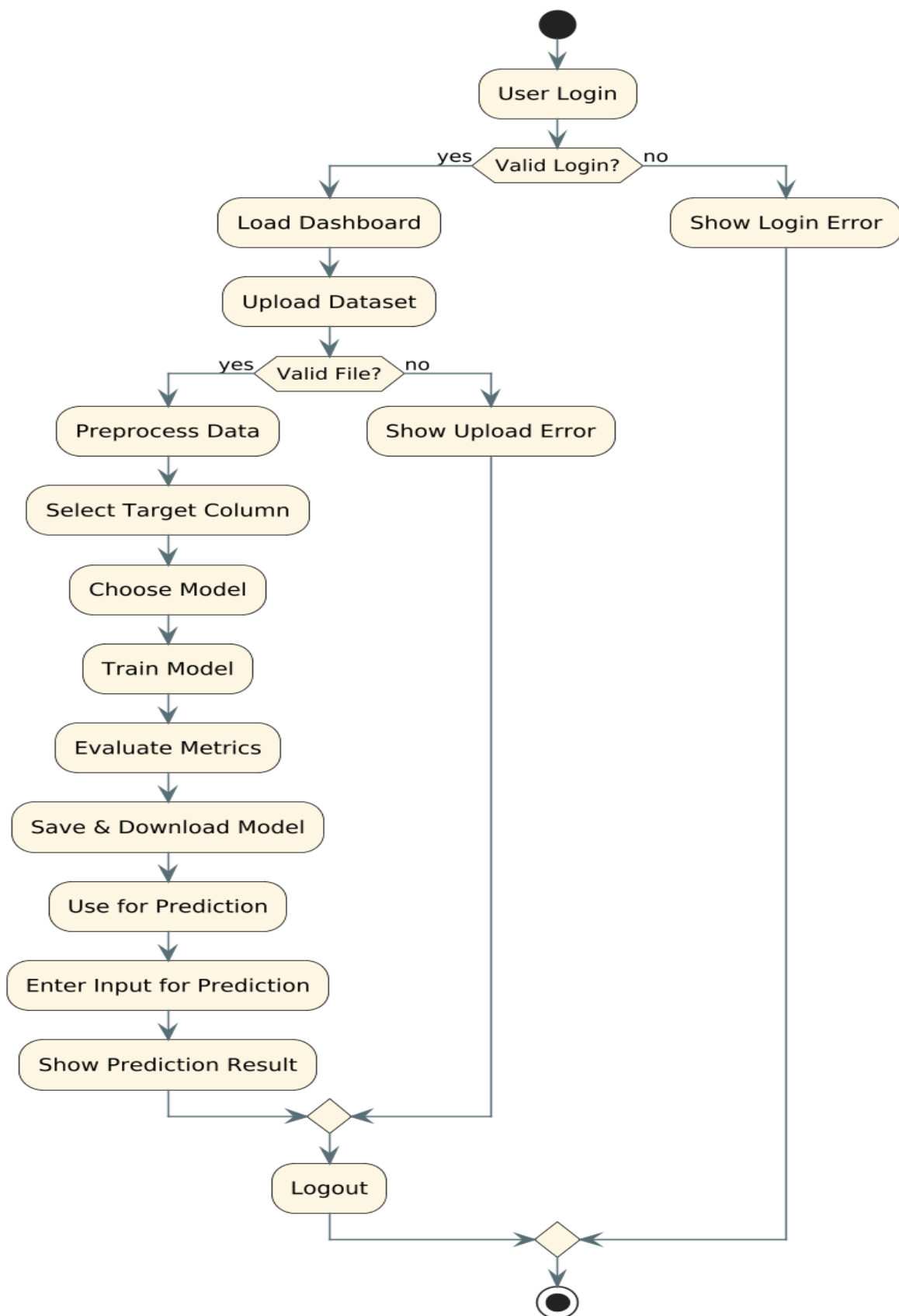


Figure 3.4.4 Activity Diagram

Data Flow Diagram (DFD)

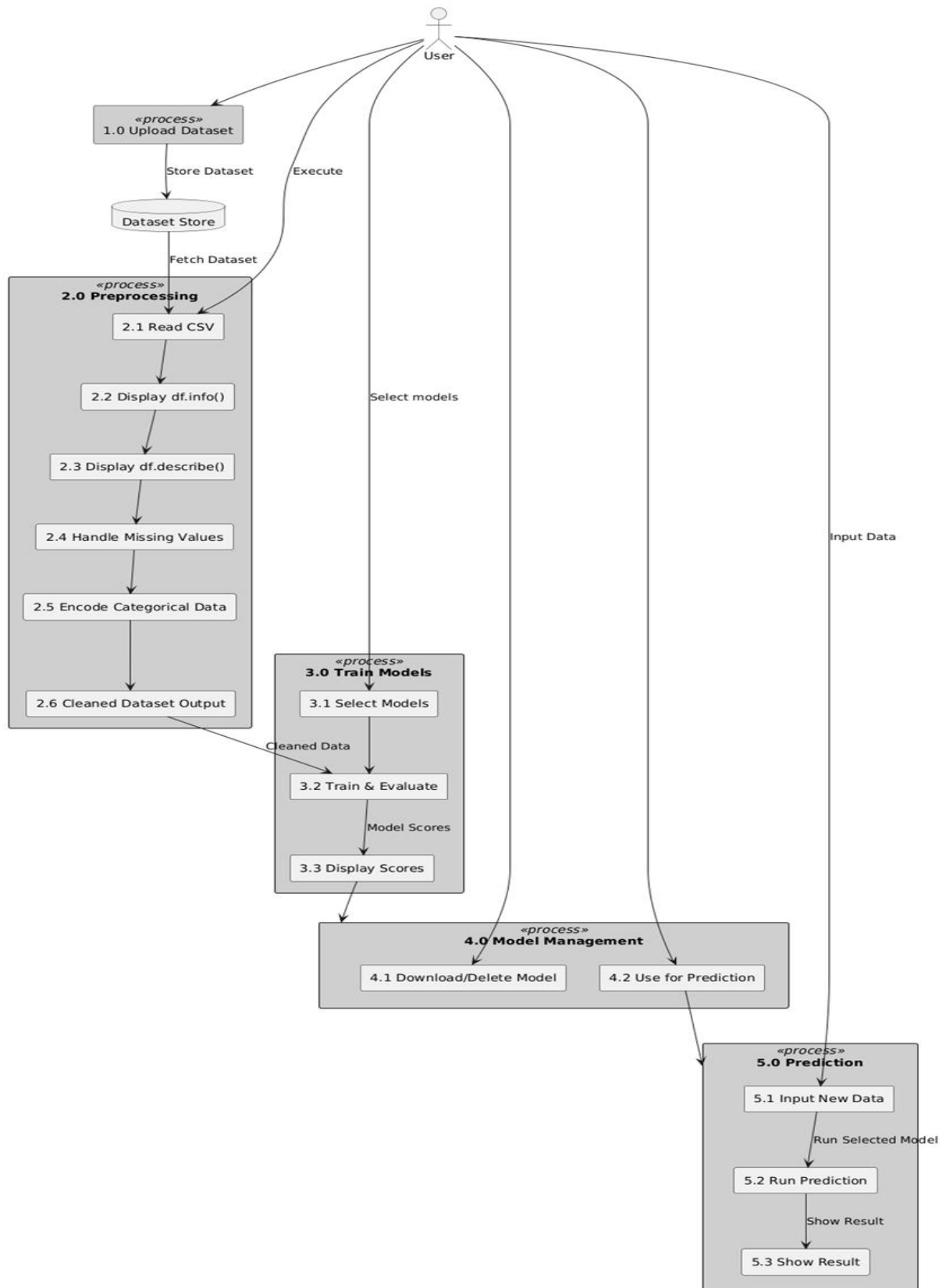


Figure 3.4.5 DFD

CHAPTER 4

IMPLEMENTATION

4.1. Frontend Implementation

The frontend of MLMCreator is developed using Streamlit, a Python library that enables easy and interactive web app creation. The focus is on creating an intuitive and seamless user interface for training machine learning models, preprocessing data, and evaluating results.

1. Streamlit Setup:

The app is organized into a sidebar with easy navigation between different sections: Home, Upload, Preprocess, Train, Models, and Predict.

2. Main Components:

- **Sidebar Navigation:** The sidebar provides access to major steps: uploading datasets, preprocessing, model training, and evaluation. The user can easily switch between pages and see relevant tasks in each section.
- **Interactive Widgets:** Users can select the target column, choose task types (Classification or Regression), pick models, and configure data splits using Streamlit's interactive widgets.
- **Data Display:** The dataset is previewed in a table (`st.dataframe`), enabling users to explore the data before applying any processing.

3. Visualizations:

- **Performance Metrics:** Evaluation metrics like Confusion Matrix, ROC Curve, and Feature Importance (for Classification) or Actual vs. Predicted and Residual Plots (for Regression) are displayed as interactive Plotly graphs. These visualizations provide real-time insights into model performance.
- **Charts and Tables:** After training the model, metrics such as accuracy, F1-score, and precision (for Classification) or MAE and R^2 (for Regression) are displayed in tables and visualized with charts.

4. Preprocessing:

- **Missing Data Handling:** Users can handle missing data through various options, such as dropping missing rows or imputing with the mean.
- **Categorical Encoding:** Label Encoding or One-Hot Encoding options are provided for categorical variables, allowing for seamless transformations.

5. Model Download and Prediction:

Once a model is trained, users can download it using the `st.download_button`. Predictions can be made by providing input feature values, which are processed by the trained model, and the output is displayed interactively.

5. Session Management:

The app utilizes Streamlit's `st.session_state` to store user-specific data (datasets, models, etc.) across sessions. It ensures that users can seamlessly transition between different pages without losing progress.

6. Responsive Layout:

The layout is designed to be responsive, adapting to different screen sizes. The user interface is clean, intuitive, and focused on the task at hand, enhancing the overall experience.

7. Error Handling:

Clear feedback is provided to users through `st.warning`, `st.info`, and `st.error` messages, guiding them through potential issues and helping resolve them.

4.2. Backend Implementation

1. Database Integration:

The backend leverages a PostgreSQL database to store user data, datasets, model configurations, and evaluation metrics. The tables in the database include:

- Users: Stores user information (ID, credentials).
- User Datasets: Stores dataset metadata, including dataset IDs, user associations, and file paths.
- Models: Stores trained model metadata, including model names, task types, and file paths.
- Classification and Regression Metrics: Stores evaluation metrics such as accuracy, precision, recall, F1-score for classification models, and MAE, MSE, RMSE, and R^2 for regression models.

2. Model Training:

The backend handles the training of models through several pre-defined machine learning algorithms, using scikit-learn and Keras for training both traditional machine learning models and neural networks.

- Traditional Models: The backend supports a variety of models, such as Random Forest, Logistic Regression, SVM, KNN, Linear Regression, and Gradient Boosting. Users can select the desired model, and the backend trains the model using the selected dataset and configuration.

3. Data Preprocessing:

The backend includes functions for preprocessing the uploaded datasets. These functions handle:

- Handling Missing Data: Removing or imputing missing values.
- Categorical Encoding: Supporting both Label Encoding and One-Hot Encoding for categorical variables.
- Feature Scaling: Normalizing or standardizing features to ensure the models perform optimally.

4. Model Evaluation:

Once the model is trained, the backend evaluates its performance using a set of metrics based on the task type (Classification or Regression).

- For Classification: Metrics like accuracy, precision, recall, F1-score, confusion matrix, and ROC curve are computed and visualized.
- For Regression: Metrics like MAE, MSE, RMSE, and R^2 are calculated, and the results are displayed as scatter plots and residual plots.

The evaluation results are stored in the database, allowing users to compare models trained on different datasets and tasks.

4. Model Persistence:

After training and evaluation, the model is saved to the local system using Joblib, which allows for easy serialization of models. The model file is then uploaded to the database, and its metadata is updated, allowing the user to download the trained model later.

5. Session Management:

The app utilizes Streamlit's `st.session_state` to manage the user session. This ensures that datasets, trained models, and other information are retained across interactions. Each user's progress is stored independently, preventing data loss between different stages of the machine learning workflow.

6. Error Handling:

Backend operations include error handling mechanisms for database connections, model training errors, and data issues. For example, when a model training fails, appropriate error messages are displayed to users, guiding them on how to resolve issues.

7. Task Recommendation:

The backend also includes a task recommendation engine that suggests whether a model should be used for classification or regression based on the properties of the target column in the dataset. This helps users make informed decisions without needing in-depth knowledge of the task type.

8. Security and Data Integrity:

The backend ensures security by using parameterized SQL queries to prevent SQL injection and ensures data integrity through rigorous validation before inserting or updating records in the database. User data is stored securely, and access is managed through session-based authentication, ensuring that only authorized users can access or modify their data and models.

CHAPTER 5

TESTING

5.1. Unit Testing

Unit testing is a crucial part of MLMCreator's development to ensure the reliability of individual components. Each module from user authentication to dataset upload, preprocessing, model training, and evaluation was tested in isolation using Python's unit test and pytest frameworks. Backend API endpoints were tested with mock data to confirm expected outputs and status codes. For instance, tests verified file upload functionality, correct handling of empty or invalid CSVs, preprocessing operations (like null removal and encoding), and model training success with expected metrics returned. Edge cases, such as missing target selection or unsupported file formats, were also tested. Each test is automated and runs during deployment via CI/CD tools to prevent regressions. Unit testing helps maintain the accuracy and integrity of operations while facilitating quick debugging and updates.

Test Case	Module	Input	Expected Output	Pass/Fail
User login with correct credentials	Auth	Valid email & password	HTTP 200 + session start	✓ Pass
Upload valid CSV	Upload	CSV file with headers	Upload success message	✓ Pass
Upload invalid file format	Upload	TXT file	Error: Unsupported format	✓ Pass
Preprocess: Drop nulls	Preprocess	DataFrame with nulls	Cleaned DataFrame	✓ Pass
Model training: Random Forest (default)	Train	Dataset + target	Accuracy $\geq$ threshold	✓ Pass
Save model and metrics	Train/Database	Trained model metadata	DB insert success	✓ Pass
Fetch evaluation metrics	Models	Trained model ID	JSON with metrics	✓ Pass

5.2. Functional Testing

Functional testing ensures that MLMCreator's features work according to the specified requirements from the user perspective. Each functional module login system, CSV upload, preprocessing, model training, and result display was tested end-to-end through simulated user interactions. Tools like Selenium (for UI automation) and Postman (for API endpoints) were used to validate system behaviour under expected conditions. For example, uploading a valid CSV should enable preprocessing options, while skipping target selection should display a validation error. The flow from dataset upload → preprocessing → model training → viewing metrics and downloading models was tested to ensure smooth navigation and data consistency across steps. Special attention was given to validating user inputs, error messages, and feedback popups. Functional testing ensures that MLMCreator performs all tasks from user registration to model export exactly as described in its functional specifications.

Test Case	Function	Input	Expected Output	Pass/Fail
Login with valid credentials	Authentication	Email: user@test.com, Password	Redirect to Home page	✔ Pass
Upload valid CSV and display preview	Upload Module	CSV file	Table preview of data	✔ Pass
Select target and preprocess null values	Preprocess	Column with nulls	Cleaned data table	✔ Pass
Train classification model and view metrics	Train	Selected model + target	Accuracy, Confusion Matrix, etc.	✔ Pass
Download trained model	Model Management	Model ID	Model file downloaded (.pkl)	✔ Pass
Register new user with existing email	Registration	Duplicate email	Error message: Email already exists	✔ Pass
Prevent training without selecting a target	Validation	No target column selected	Alert message shown	✔ Pass

5.3. Integration Testing

Integration testing in the MLMCreator web application ensures that individual modules work together as expected when combined. After verifying that each module functions independently through unit and functional testing, integration testing focused on the data flow and interactions between components such as the frontend (Streamlit), backend (Flask APIs), and the PostgreSQL database. For example, when a user uploads a dataset, the backend must store it correctly and allow preprocessing to start. Similarly, once a model is trained, it should be saved in the database and listed under the Models page. Integration testing validates sessions, persistence, and transitions between Upload → Preprocessing → Training → Results. Tools like pytest, unit test (Python), and Postman were used to test API sequences, while manual and automated testing confirmed UI-to-backend sync. Successful integration testing ensures that MLMCreator functions cohesively and provides a smooth, stepwise ML workflow experience to users.

Test Case	Modules Involved	Input	Expected Output	Result
IT-01: Upload to Preprocess Flow	Upload → Preprocess	Valid CSV file	File uploaded and preprocessed successfully	 Pass
IT-02: Preprocess to Train Flow	Preprocess → Train	Cleaned dataset with no nulls	Model training initiated	 Pass
IT-03: Train to Models DB Storage	Train → Save to Database	Trained model + metrics	Model and metrics saved	 Pass
IT-04: Session State Sync	All pages	User session with uploaded data	Dataset state consistent across pages	 Pass
IT-05: Model Retrieval and Display	Models → PostgreSQL	User login and selects past model	Model metrics and details loaded successfully	 Pass

5.4. System Testing

System testing is conducted to verify that the entire MLMCreator web application functions correctly as a complete and integrated system. This phase validates both the functional and non-functional requirements of the application in a real-world environment. It ensures the seamless interaction between modules such as user authentication, dataset upload, preprocessing, model training, and database storage. The testing process simulates end-user behaviour, verifying correct flow across the system, session persistence, proper error handling, and data integrity. Tools like Postman (for API testing), pgAdmin (for DB checks), and browser-based manual testing were used to execute full workflow scenarios. The goal of system testing is to ensure that the product meets its intended requirements, delivers a smooth user experience, and maintains stability and performance across all expected use cases.

Test Case				
ID	Test Scenario	Input	Expected Output	Result
ST-01	End-to-End Workflow	Login → Upload → Train → Save	Successful model creation and storage	 Pass
ST-02	Invalid Dataset Handling	Malformed CSV	Error message displayed	 Pass
ST-03	Session Expiry	Inactivity for 10 minutes	Redirect to login page	 Pass
ST-04	Model Comparison	Select multiple models	Comparison metrics and graphs displayed	 Pass
ST-05	Database Consistency	Check DB after save and retrieval	Same data structure and metrics preserved	 Pass

CHAPTER 6

CONCLUSION

The MLMCreator app is a comprehensive, user-friendly tool designed to simplify the process of building and deploying machine learning models. By integrating a wide range of machine learning algorithms, data preprocessing tools, and evaluation metrics, it provides users with an efficient and effective platform for their model development needs. The app's functionality is enhanced with a robust backend that supports both traditional machine learning models and deep learning models, such as Convolutional Neural Networks (CNNs), which expands its applicability to image-based tasks.

Key features, such as the automated task type recommendation based on dataset characteristics, customizable train/test split ratios, and seamless model training and evaluation, empower users to build and assess models without requiring deep technical expertise. The app also supports detailed performance metrics visualization and feature importance analysis, helping users understand their models' behaviour and identify areas for improvement.

Through careful integration with a PostgreSQL database, the app ensures data persistence, secure model storage, and easy access to evaluation results. Users can save their trained models and corresponding metrics, enabling them to revisit, download, or share their models at any time.

From a technical perspective, MLMCreator combines the power of popular machine learning libraries, like scikit-learn and Keras, with the ease of use and interactive interface provided by Streamlit. The integration of session management ensures a personalized and uninterrupted user experience, making it suitable for both novice and experienced data scientists.

In conclusion, MLMCreator is not just a tool for model training but an end-to-end solution that democratizes machine learning for all users, from data preprocessing to model deployment. It makes the process of building, evaluating, and storing machine learning models more accessible and efficient, ensuring that users can focus on deriving insights and making data-driven decisions. This platform is poised to be an asset for individuals and organizations looking to leverage machine learning for various tasks across industries.

6.1. Foreseeable Enhancements

1. Support for Additional Dataset Types

While the current version focuses on tabular data, future enhancements could include support for other data types such as images, text, and time-series data. This would allow broader use cases like NLP and computer vision tasks within the same framework.

2. AutoML Integration

Incorporating AutoML tools (such as H2O.ai, TPOT, or auto-sklearn) can help automate model selection, feature engineering, and hyperparameter tuning. This would be highly beneficial for non-technical users and accelerate the training pipeline.

3. Advanced Visualization Dashboard

Currently, the app provides basic graphs. Future versions could integrate advanced visualization libraries like Plotly Dash or Streamlit components to offer interactive dashboards, drill-down analytics, and real-time performance tracking.

4. Model Deployment & API Generation

The app can be enhanced to allow one-click deployment of trained models as REST APIs using tools like Flask, FastAPI, or Streamlit sharing. This would make it easy to consume the trained models in external applications.

5. Custom Model Upload

Allowing users to upload their own pre-trained scikit-learn or pickle models (.pkl) would make the tool more flexible and user-driven.

6. Role-Based Access Control

Enhancing the authentication system to include roles such as Admin, Analyst, and Viewer could ensure secure access and streamline workflows in team environments.

7. Cloud Storage Integration

Adding support for cloud storage services like Google Drive, AWS S3, or Dropbox would enable users to load and save datasets/models seamlessly from cloud environments.

APPENDIX A - CODES

App.py

```
# app.py
import streamlit as st
from streamlit_option_menu import option_menu
import time
import importlib
st.set_page_config(page_title="MLMCreator", layout="wide")
PAGE_TITLES = {
    "Home": "page.home",
    "Upload": "page.upload",
    "Pre-Process": "page.preprocess",
    "Train": "page.train",
    "Models": "page.models",
    "Predict": "page.predict"
}
if "authenticated" not in st.session_state:
    st.session_state["authenticated"] = False
if "user_id" not in st.session_state:
    st.session_state["user_id"] = None
if "last_activity" not in st.session_state:
    st.session_state["last_activity"] = time.time()
if st.session_state["authenticated"]:
    if time.time() - st.session_state["last_activity"] > 1800:
        st.session_state["authenticated"] = False
        st.session_state["user_id"] = None
        st.warning("Session expired. Please log in again.")
        st.stop()
    else:
        st.session_state["last_activity"] = time.time()
with st.sidebar:
    st.session_state.selected = option_menu(
        menu_title=None,
        options=["Home", "Upload", "Pre-Process", "Train",
"Models", "Predict"],
        icons=["house", "cloud-upload", "gear", "bar-chart", "folder", ""],
        default_index=0
    )
    st.markdown("---")
    if st.button("Logout"):
        st.session_state["authenticated"] = False
        st.session_state["user_id"] = None
        st.rerun()
if st.session_state.selected in PAGE_TITLES:
    module = importlib.import_module(PAGE_TITLES[st.session_state.selected])
    module.render()
```

Home.py

```
# home.py
import streamlit as st
import psycopg2
import bcrypt
from database.db import get_connection
import importlib

def render():
    st.title(" MLMCreator")
    st.subheader("End-to-end machine learning model creation made easy.")

    if st.session_state.get("authenticated"):
        st.text("Already Logged in, Please carry on")
        return

    if "show_register" not in st.session_state:
        st.session_state["show_register"] = False

    if st.session_state["show_register"]:
        render_register()
    else:
        render_login()

def render_login():
    st.header("Login")
    email = st.text_input("Email")
    password = st.text_input("Password", type="password")

    if st.button("Login"):
        if not email or not password:
            st.error("Please fill all fields.")
            return

        conn = get_connection()
        cur = conn.cursor()
        cur.execute("SELECT id, password FROM users WHERE email = %s",
(email,))
        user = cur.fetchone()
        cur.close()
        conn.close()
        if user and bcrypt.checkpw(password.encode(), user[1].encode()):
            st.session_state["authenticated"] = True
            st.session_state["user_id"] = user[0]
            st.session_state.df=None
            st.session_state['dataset_id']=None
            st.success("Login successful.")
        else:
            st.error("Invalid credentials.")
```

```

    if st.button("Register New Account"):
        st.session_state["show_register"] = True

def render_register():
    st.header("Register")
    email = st.text_input("Email")
    password = st.text_input("Create Password", type="password")

    if st.button("Create Account"):
        if not email or not password:
            st.error("Please fill all fields.")
            return

        hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt()).decode()
        try:
            conn = get_connection()
            cur = conn.cursor()
            cur.execute("INSERT INTO users (email, password) VALUES (%s, %s)",
(email, hashed))
            conn.commit()
            cur.close()
            conn.close()
            st.success("Account created. Please login.")
            st.session_state["show_register"] = False
        except psycopg2.Error:
            st.error("Email already registered.")

    if st.button("Back to Login"):
        st.session_state["show_register"] = False

```

APPENDIX B - SCREEN SHOTS

Home Page

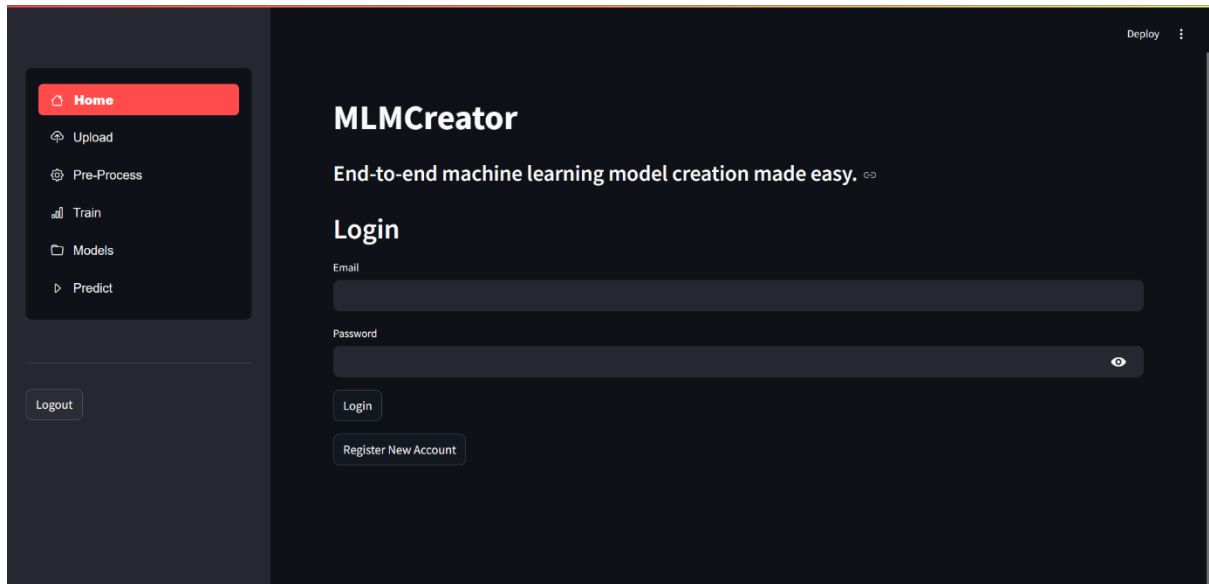


Figure 1 Home Page

Upload Page

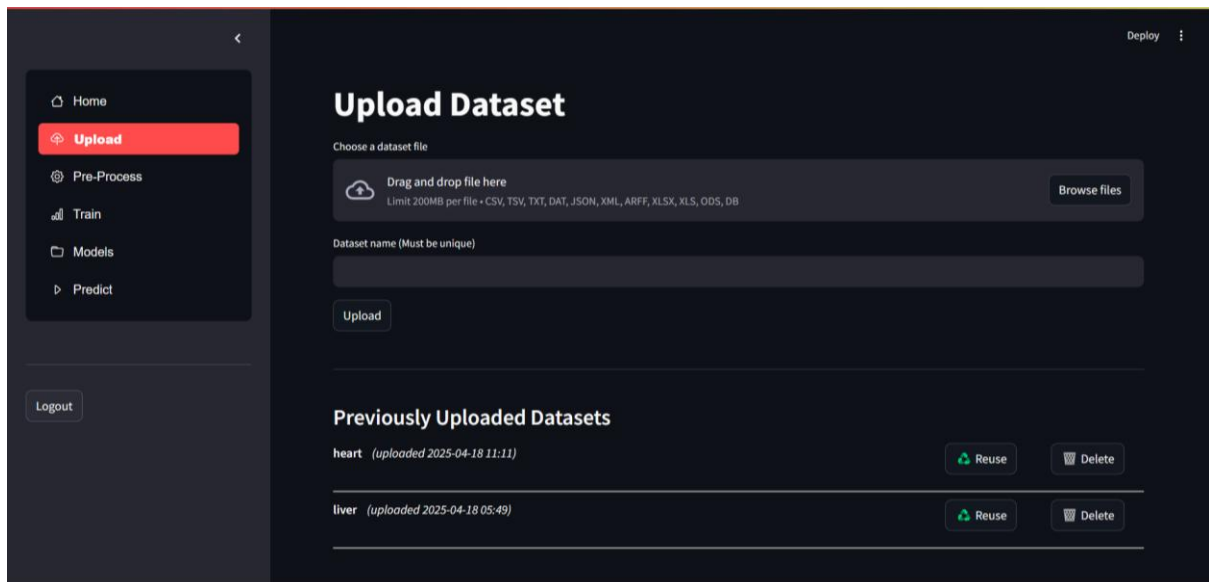
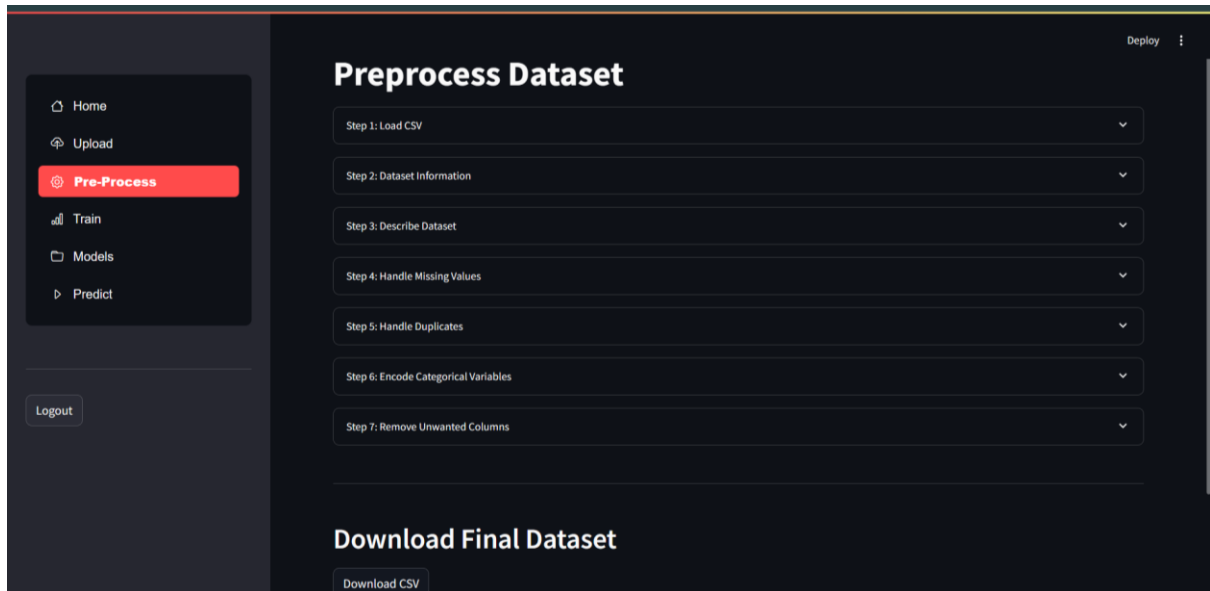


Figure 2 Upload Page

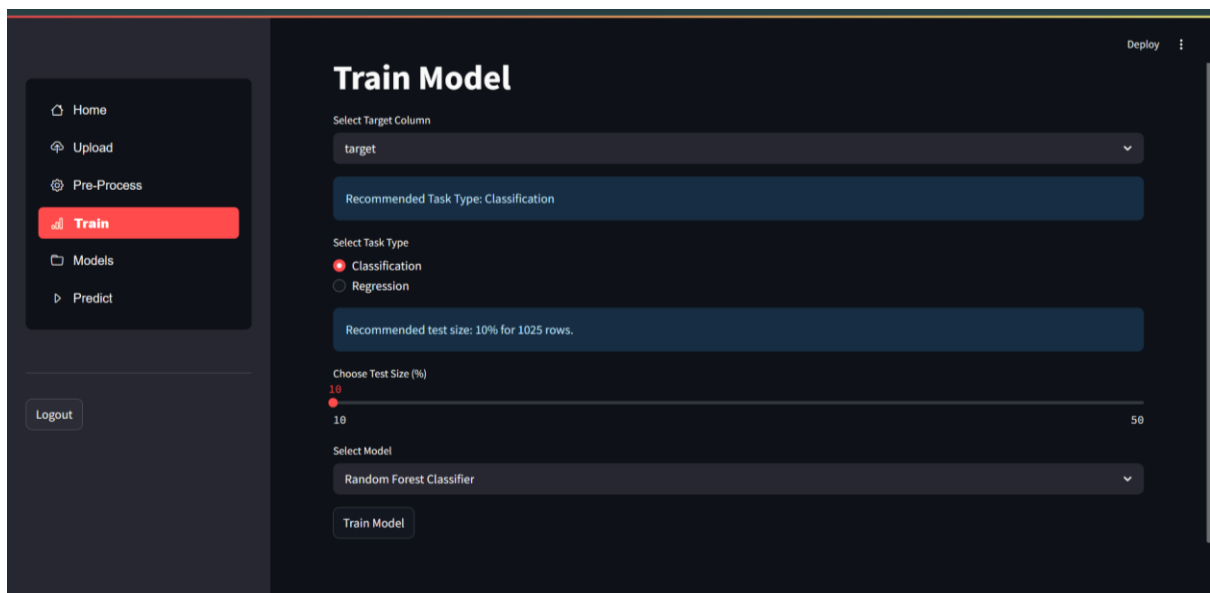
Pre-process Page



The screenshot shows the 'Preprocess Dataset' page. On the left is a sidebar with navigation links: Home, Upload, Pre-Process (highlighted in red), Train, Models, and Predict. Below these is a 'Logout' button. The main content area has a 'Deploy' button in the top right. The title 'Preprocess Dataset' is at the top. Below it is a list of seven steps, each in a dropdown menu: Step 1: Load CSV, Step 2: Dataset Information, Step 3: Describe Dataset, Step 4: Handle Missing Values, Step 5: Handle Duplicates, Step 6: Encode Categorical Variables, and Step 7: Remove Unwanted Columns. At the bottom, there is a section titled 'Download Final Dataset' with a 'Download CSV' button.

Figure 3 Pre-process Page

Train page



The screenshot shows the 'Train Model' page. The sidebar is identical to the previous page, but 'Train' is highlighted in red. The main content area has a 'Deploy' button in the top right. The title 'Train Model' is at the top. Below it is a 'Select Target Column' dropdown menu with 'target' selected. A blue box displays 'Recommended Task Type: Classification'. Under 'Select Task Type', 'Classification' is selected with a red radio button, and 'Regression' is unselected. Another blue box shows 'Recommended test size: 10% for 1025 rows.' Below this is a 'Choose Test Size (%)' slider with markers at 10, 10, and 50. The 'Select Model' dropdown menu has 'Random Forest Classifier' selected. At the bottom is a 'Train Model' button.

Figure 4 Train Page

Models Page

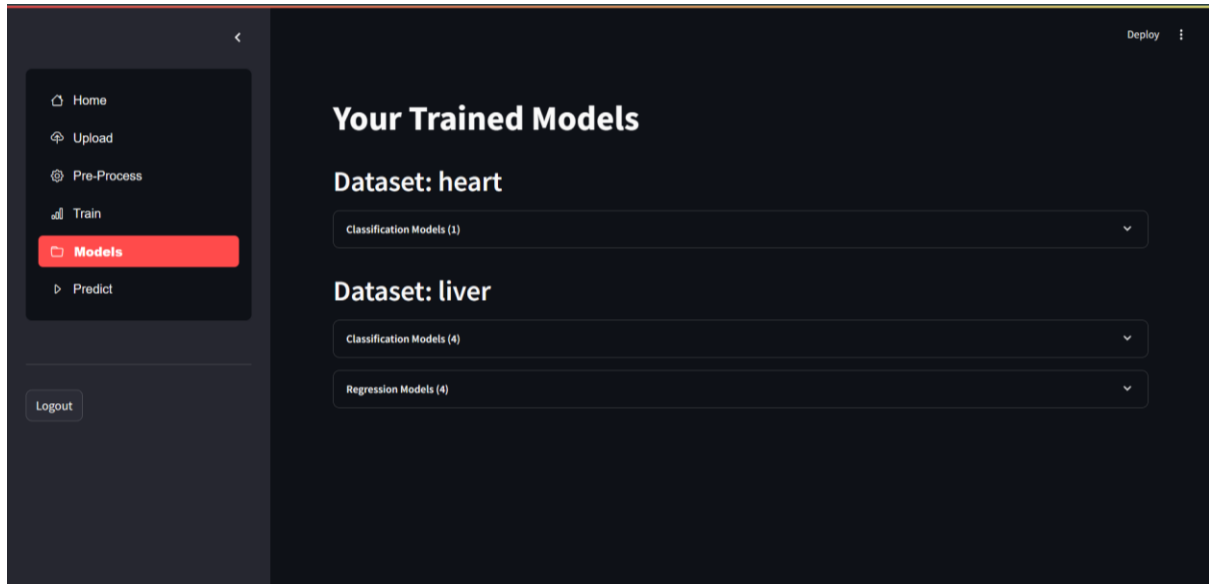


Figure 5 Models Page

Predict page

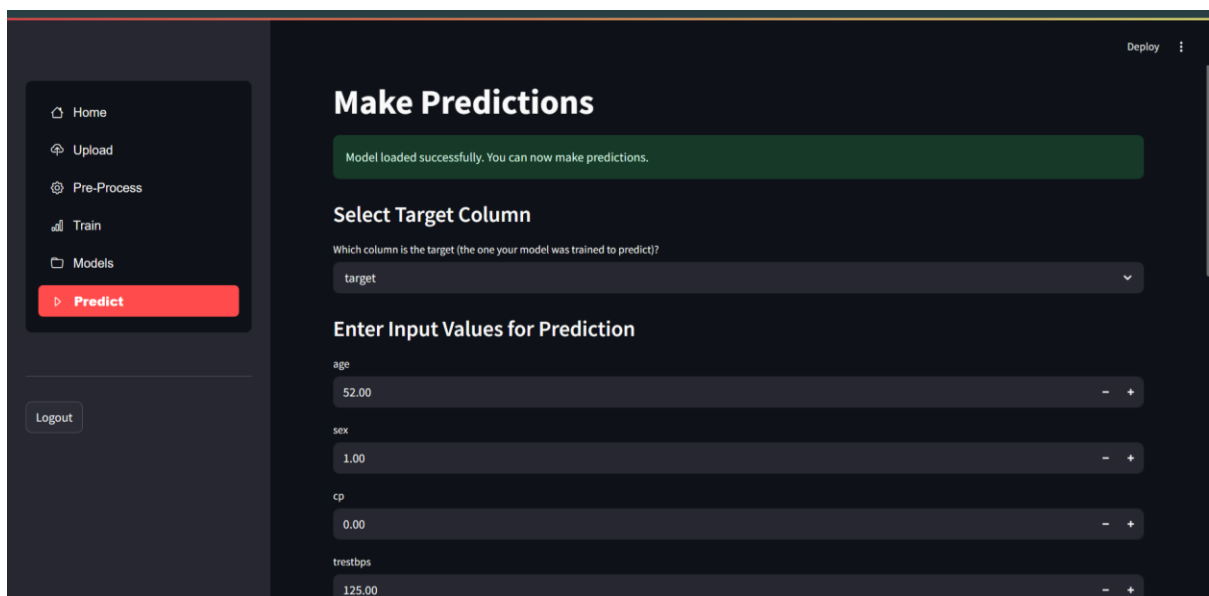


Figure 6 Predict Page

BIBLIOGRAPHY

1. Buitinck, L., Louppe, G., Blondel, M., Pedregosa, F., Mueller, A., Grisel, O., ... & Varoquaux, G. (2013). API design for machine learning software: Experiences from the scikit-learn project. *Proceedings of the 2013 Python in Science Conference*, 108–115.
2. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. <https://doi.org/10.1145/2939672.2939785>
3. Cockcroft, M., & Drummond, C. (2020). Streamlit: Turn data scripts into shareable web apps in minutes. *Proceedings of the 2020 Conference on Innovative Data Systems Research*.
4. Dunning, T., & Friedman, E. (2014). *Practical Data Wrangling*. O'Reilly Media.
5. Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., ... & Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
6. McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 51–56.
7. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
8. Pondicherry University. (2025). *Master of Computer Applications curriculum*. Pondicherry University. (Context for academic background.)
9. Stonebraker, M., & Rowe, L. (1986). The design of POSTGRES. *Proceedings of the 1986 ACM SIGMOD International Conference on Management of Data*, 340–355.
10. Wrembel, R., & Koncilia, C. (2019). Data preprocessing techniques for classification without discrimination. *Information Systems*, 84, 165–179. <https://doi.org/10.1016/j.is.2019.02.001>